

# Linux

BB1000 Programming in Python KTH

# First essentials

Make a directory: `mkdir MY_FIRST_DIRECTORY` Create an empty file: `touch my_first_file` List all the files (and directories) in the current directory: `ls` Go into the directory `MY_FIRST_DIRECTORY`: `cd MY_FIRST_DIRECTORY` To show in which directory the cursor is located: `pwd` Go out of the directory: `cd ..`

```
$ pwd
/home/kronop
$ ls
$ mkdir MY_FIRST_DIRECTORY
$ ls
examples.desktop MY_FIRST_DIRECTORY
$ cd MY_FIRST_DIRECTORY
$ touch my_first_file
$ ls
my_first_file
$ pwd
/home/kronop/MY_FIRST_DIRECTORY
$ cd ..
$ ls
examples.desktop MY_FIRST_DIRECTORY
```

To remove an empty directory, `rmdir` is used. Files are deleted by `rm`.

Remark: for a python input cursor, we use `>>>` in this course - for a linux input cursor, we use `$`.

# Move files

To rename files and to move files to another directory, the same keyword `mv` is used.

```
$ cd MY_FIRST_DIRECTORY
$ mv my_first_file my_first_other_name
$ ls
my_first_other_name
$ mv my_first_other_name ../
$ cd ..
$ pwd
/home/kronop
$ ls
examples.desktop MY_FIRST_DIRECTORY my_first_other_name
$ mv my_first_other_name MY_FIRST_DIRECTORY/my_first_file
$ ls MY_FIRST_DIRECTORY
my_first_file
```

# Copy files

Files are copied using `cp` - remark that the user always has to specify the destination...

```
$ cd MY_FIRST_DIRECTORY
$ cp my_first_file copy_of_my_first_file
$ ls
my_first_file copy_of_my_first_file
$ cp copy_of_my_first_file ../
$ cd ..
$ pwd
/home/kronop
$ ls
examples.desktop MY_FIRST_DIRECTORY copy_of_my_first_file
$ ls MY_FIRST_DIRECTORY
my_first_file copy_of_my_first_file
```

# Remove files

Keyword: `rm`...

```
$ pwd
/home/kronop
$ rm copy_of_my_first_file MY_FIRST_DIRECTORY/copy_of_my_first_file
$ ls
examples.desktop MY_FIRST_DIRECTORY
$ ls MY_FIRST_DIRECTORY
$ my_first_file
```

# "Hidden" files

Everything is governed by a text-file... Understand which file describes what!

With `ls -alh` all files in the directory are given - the normal files as well as the hidden ones which start with a `.`. The files are listed with their permissions, the name of their owner and the group of the owner.

```
$ ls -alh
drwxr-xr-x  3 kronop sknippen 4,0K nov 17  2016 .
drwxr-xr-x 31 root    root    4,0K feb  4 20:54 ..
-rw-----  1 kronop sknippen   59 nov 17  2016 .bash_history
-rw-r--r--  1 kronop sknippen  220 nov 17  2016 .bash_logout
-rw-r--r--  1 kronop sknippen 3,6K nov 17  2016 .bashrc
drwx-----  2 kronop sknippen 4,0K nov 17  2016 .cache
-rw-r--r--  1 kronop sknippen 8,8K nov 17  2016 examples.desktop
drwxr-xr-x  2 kronop sknippen 4,0K feb 11 21:39 MY_FIRST_DIRECTORY
```

The response above can be compared with

```
$ ls
examples.desktop MY_FIRST_DIRECTORY
```

User kronop is a member of the group around user sknippen. The date of creation of the files is given (17 nov 2016), as well as the extend of the files (in Bytes).

# Permissions

The permissions of the files (and directories if the first symbol is a `d`) are given in the first column. The first three symbols denote the usage for the owner of the file: `rw` points at reading, writing and executing. The following three symbols denote the use for members of the group sknippen, of which the user kronop is a member.

In many cases, the group members are allowed to read the files (`r`) and eventually execute them (`x`). The last three digits denote the rights for users on the clusters which do not belong to the group sknippen. In any case, a normal user would not allow other people to change his/her files, so writing (`w`) is advised to be disabled for groupmembers and certainly for other people.

When a program is written, it is actually a text-file, which is made executable (`x`).

```
$ touch my_first_empty_program
$ ls -alh my_first_empty_program
-rw-r--r-- 1 kronop sknippen 0 feb 11 21:54 my_first_empty_program
```

# Change permissions

The file `my_first_empty_program` is readable and writable for `kronop`, and it is just readable for the members of the group `sknippen` and for the rest of the users. It is not yet executable, therefore `chmod` is used.

`chmod u+x` renders the file for the user `kronop` to an executable file. `chmod g+x` gives also the members of the group the opportunity to run the file.

```
$ chmod u+x my_first_empty_program
$ ls -alh my_first_empty_program
-rwxr--r-- 1 kronop sknippen 0 feb 11 21:54 my_first_empty_program
$ chmod g+x my_first_empty_program
$ ls -alh my_first_empty_program
-rwxr-xr-- 1 kronop sknippen 0 feb 11 21:54 my_first_empty_program
```

# Settings

In the `.bashrc` file, the personalized settings for the user `kronop` are defined.

From a cautious point of view, important settings could be:

```
alias rm="rm -i"  
alias cp="cp -i"  
alias mv="mv -i"
```

Thanks to these settings, linux will ask you whether you would really like to remove (`rm`) or overwrite (in case of `cp` and `mv`) files.

In this `.bashrc` script, packages like e.g. compilers are also loaded:

```
source /opt/intel/bin/iccvars.sh intel64  
source /opt/intel/bin/compilervars.sh intel64
```

In case you write your own programs, you have to tell linux where they are located. Generally, a typical user like `kronop` has the habit to locate them in the `/home/kronop/bin` folder, which has to be loaded in the `.bashrc` file:

```
export PATH=/home/kronop/bin:$PATH
```

## When the `.bashrc` file has been changed...

... you have to reload or `source` it so that the changes can take effect:

```
$ pwd
/home/kronop
$ source .bashrc
$
```

# Text editor

In linux, many plain text-editors are available: `nano`, `gedit`,... However, probably the most widely used ones are `vi` and `emacs`.

`$ emacs filename` will open a window in which the user can type plain text (a bit comparable with wordpad in Windows). To save the file, two times two keys has to be used: 'ctrl-x ctrl-s'. To close the file, 'ctrl-x ctrl-c' has to be typed. To cut and paste, 'ctrl-k' and 'ctrl-y' are used, respectively. Searching a word throughout the file is done using 'ctrl-s'.

When `$ vi filename` is chosen, the user has to type first an `i` in order to be able to insert, change or type plain text. When he/she is ready, `esc` has to be pressed, followed by `:wq!` to close the file. To just close the file without saving the changes, `:q!` has to be prompted.

# Finding files

It is a general advice to keep all your files ordered. It might be easy at this moment to find files back, but within three months, you should still know which file was connected with what project and how it was created...

However, to search files `find ./ -name` is used. The `./` points at the directory where (and down) the file is searched.

```
$ pwd
/home/kronop
$ find ./ -name my_first_file
./MY_FIRST_DIRECTORY/my_first_file
```

# Searching information in a file

Imagine user kronop has to search the birth date of a friend Erik Ohlsson in a file which lists all the inhabitants of the village with their data.

```
$ cat birthdates_village
Olav Carlsson 1969-01-12
Emma Karlsson 1981-04-23
Erik Ohlsson 1972-12-3
Mohammed Puidgemon 1988-02-29
Helen Tantra 1945-09-15
(...)
```

`cat` prints the content of a file to screen - but it cannot be changed.

In stead of browsing through the file, the linux function `grep` can be used.

```
$ grep Erik birthdates_village
Erik Ohlsson 1972-12-3
```

# Changing information in a file

The family Ohlsson changes name into Bernadotte. The register of the village has to be adapted. The function `sed -i s/something/somethingelse/g filename` is used: in 'filename', each 'something' is changed into 'somethingelse'.

```
$ sed -i s/Ohlsson/Bernadotte/g birthdates_village
$ grep Erik birthdates_village
Erik Bernadotte 1972-12-3
```

# What is the structure among my files?

At each instance in the hierarchy of the data, keyword `tree` can show the structure of the directories and the files at that position and below.

```
$ pwd  
/home/kronop  
$ tree
```

```
├── examples.desktop  
├── MY_FIRST_DIRECTORY  
│   ├── birthdates_village  
│   ├── my_first_empty_program  
│   └── my_first_file  
└── my_first_program.py  
1 directory, 5 files
```

# When you don't know...

Linux has a built-in help function, which is especially useful when you do not know which optional parameters can be used... Simply write `man` followed by the command you would like to have more information about. `man ls` will give various pages of information about the `ls` command...

```
$ man ls
```

```
LS(1)                                User Commands                                LS(1)
```

```
NAME
```

```
ls - list directory contents
```

```
SYNOPSIS
```

```
ls [OPTION]... [FILE]...
```

```
DESCRIPTION
```

```
List information about the FILES (the current directory by default).  
(...)
```

# Which jobs are running?

To know his own jobs which are (still) running on a server, user kronop uses `ps -fu kronop`.

```
$ ps -fu kronop
UID      PID  PPID  C  STIME TTY          TIME CMD
kronop   4411  4358  0  11:46 ?        00:00:00 sshd: kronop@pts/14
kronop   4412  4411  0  11:46 pts/14    00:00:00 -bash
kronop   4586  4532  0  11:49 ?        00:00:00 sshd: kronop@pts/16
kronop   4587  4586  0  11:49 pts/16    00:00:00 -bash
kronop   4641  4412  0  11:51 pts/14    00:00:00 python3 my_first_program.py
kronop   4642  4587  0  11:51 pts/16    00:00:00 ps -fu kronop
```

User kronop is running a python program 'my\_first\_program'. The careful reader can also see that the command gives a list of jobs, which somehow depend on each other: task 4641 is connected to 4412, therefore to 4411, which is the connection to the machine made using sshd at 11h46. In this particular case, it is also seen that the `ps -fu kronop` command is prompt to another window which was opened at 11h49.

# Which jobs are running?

Each user can see the list of all currently running jobs on a server using `top`.

The output is then

```
Tasks: 333 total, 2 running, 330 sleeping, 1 stopped, 0 zombie
%Cpu(s): 20,0 us, 0,0 sy, 0,0 ni, 79,9 id, 0,0 wa, 0,0 hi, 0,0 si, 0,0 st
KiB Mem: 19804076+total, 13259345+used, 65447304 free, 1075368 buffers
KiB Swap: 19531264+total, 72036 used, 19524060+free, 12587646+cached Mem
```

```
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
5273 hforb 20 0 1398236 492784 78964 R 400,3 0,2 71:34.28 qcprog.exe
5479 kronop 20 0 30516 3440 2772 R 0,3 0,0 0:00.01 top
1 root 20 0 34704 3400 1856 S 0,0 0,0 1:12.16 init
2 root 20 0 0 0 0 S 0,0 0,0 0:00.61 kthreadd
3 root 20 0 0 0 0 S 0,0 0,0 0:01.09 ksoftirqd/0
5 root 0-20 0 0 0 S 0,0 0,0 0:00.00 kworker/0:0H
6 root 20 0 0 0 0 S 0,0 0,0 8:25.04 kworker/u40:0
8 root 20 0 0 0 0 S 0,0 0,0 21:00.97 rcu_sched
9 root 20 0 0 0 0 S 0,0 0,0 0:54.34 rcuos/0
10 root 20 0 0 0 0 S 0,0 0,0 8:31.40 rcuos/1
11 root 20 0 0 0 0 S 0,0 0,0 0:47.52 rcuos/2
12 root 20 0 0 0 0 S 0,0 0,0 0:58.93 rcuos/3
13 root 20 0 0 0 0 S 0,0 0,0 0:30.22 rcuos/4
14 root 20 0 0 0 0 S 0,0 0,0 15:10.08 rcuos/5
15 root 20 0 0 0 0 S 0,0 0,0 2:18.49 rcuos/6
16 root 20 0 0 0 0 S 0,0 0,0 1:23.19 rcuos/7
17 root 20 0 0 0 0 S 0,0 0,0 0:31.34 rcuos/8
18 root 20 0 0 0 0 S 0,0 0,0 0:50.12 rcuos/9
19 root 20 0 0 0 0 S 0,0 0,0 0:16.02 rcuos/10
20 root 20 0 0 0 0 S 0,0 0,0 1:15.81 rcuos/11
21 root 20 0 0 0 0 S 0,0 0,0 0:21.05 rcuos/12
22 root 20 0 0 0 0 S 0,0 0,0 0:21.75 rcuos/13
```

# Which jobs are running?

It is seen that user hforsb runs a job 'qcprog.exe' with 4 cores (400%) since 71 hours and user kronop is freshly running the top command.

With the 4 cores, the server is only 20% used. This means that 80% of the machine is 'idle' - the server contains out of 20 cores.